

算法设计与分析

第一章

算法的概念及分析方法

王晓茹

计算机学院



目录

CONTENTS

01

算法发展简史
与理论根基

02

算法世界初探
基本概念

03

复杂度
度量算法效率的标尺

04

渐近符号五重奏

05

复杂度估算实战
与主定理

06

算法设计范式总览
高级算法思想与前沿趋势

01

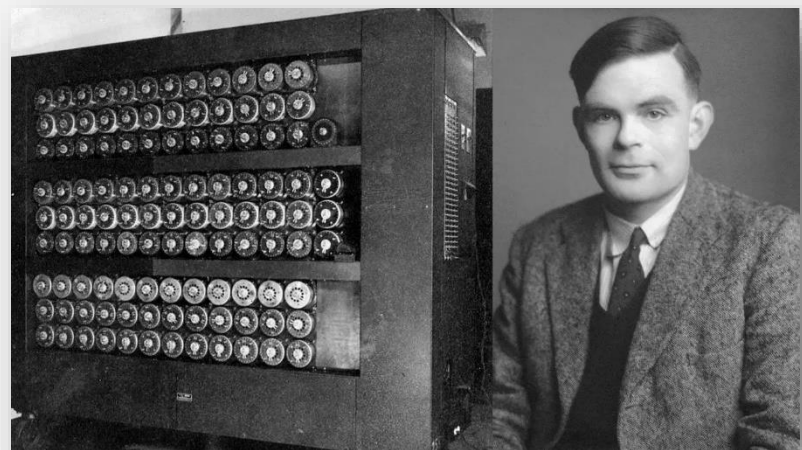
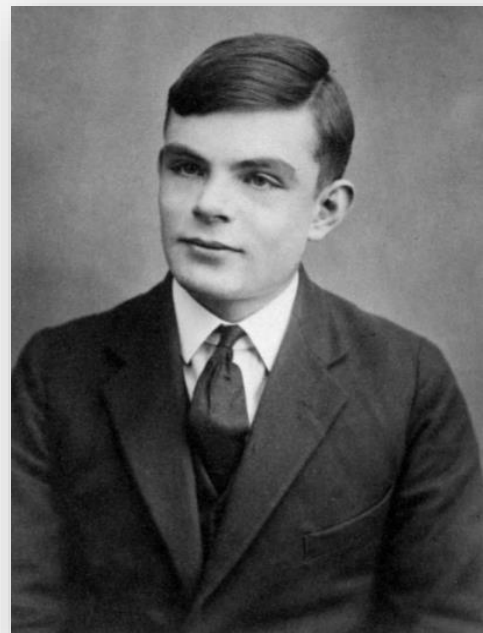
算法发展简史与理论根基



图灵机与 λ 演算：可计算性奠基

20世纪30年代，丘奇、哥德尔与图灵分别用 λ 演算、不完备定理与图灵机为“算法可计算”奠定严格定义。

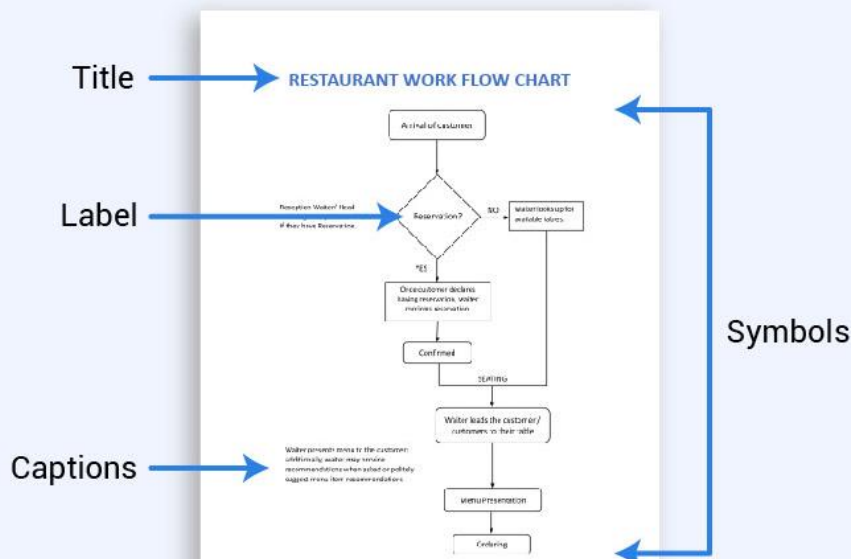
丘奇-图灵论题将三者统一，指出凡可算法求解者皆可用图灵机模拟，由此划出计算能力的理论边界，成为后续计算机科学研究共同起点。



冯·诺依曼架构：通用计算机的起点

1945年冯·诺依曼提出EDVAC方案，确立“**程序指令与数据同存**”的存储程序思想，使计算机从专用走向通用。

运算器、控制器、存储器、输入输出五大部件沿用至今，为算法在硬件上运行提供标准化舞台，催生后续软件与算法研究的繁荣。



图灵奖中的算法印记

回顾1970至2024年图灵奖，至少有十一项直接表彰算法与复杂性理论贡献，奖项轨迹映射出算法研究从经典数值到智能学习的演进脉络。

算法方面

- 1970: 詹姆斯·威尔金森 - 数值线性代数
- 1972: 埃德斯加·狄克斯特拉 - 结构化编程
- 1974: 唐纳德·克努特 - 算法分析
- 1980: 托尼·霍尔 - 快速排序算法
- 1986: 霍普克洛夫特与塔扬 - 算法与数据结构
- 2020: 阿霍与厄尔曼 - 编程语言算法
- 2024: 巴托与萨顿 - 强化学习算法

复杂性理论方面

- 1982: 斯蒂芬·库克 - NP完全性理论
- 1985: 理查德·卡普 - 算法理论与NP完全性
- 1993: 哈特马尼斯与斯特恩斯 - 计算复杂性基础
- 1995: 曼纽尔·布卢姆 - 复杂性理论与密码学

算法与社会的双向塑造



算法的社会属性与伦理底线

算法并非纯粹技术，其设计嵌入文化价值观。不同群体对隐私、公平、集体利益的权重差异，直接影响需求定义与功能取舍。

若忽视多元语境，算法落地易遭遇接受度落差，甚至 **放大社会裂痕**，故社会属性需在需求阶段即纳入考量。



公平、透明与责任：三大伦理原则



公平性

要求算法不因训练数据偏差而对种族、性别等群体产生系统性歧视，确保决策中立。



透明性

强调模型可解释，使决策过程能被监管与公众理解，建立信任。



责任性

厘清设计、开发、部署各环节主体，确保损害发生时可追溯、可问责。

算法社会的收益与风险

积极影响

- ✓ 效率提升: 智能交通系统优化流量, 减少拥堵。
- ✓ 技术创新: AI、大数据等技术发展离不开算法支持。
- ✓ 科学决策: 政府和企业可基于数据分析做出更科学的决策。

消极风险

- ⚠ 算法歧视: 人脸识别等技术存在种族、性别偏见。
- ⚠ 信息茧房: 推荐系统导致认知窄化, 观点极化。
- ⚠ 隐私侵犯: 数据收集与分析可能泄露个人隐私。

融合伦理的算法设计新模式



需求驱动

引入多样化利益相关方，兼顾不同文化背景与价值观。



技术实现

采用联邦学习、差分隐私等技术保障数据安全与公平性。



人类监督

保留Human-in-the-loop机制，确保人类对关键决策的最终裁量权。

02

算法世界初探 基本概念



算法与程序

本质差异与内在联系



算法 (Algorithm)

解决问题的方法和过程，强调解题逻辑与有限性。



程序 (Program)

算法用编程语言的具体实现，侧重实现细节。

算法是 **解决问题的有限指令序列**，而程序是算法在特定编程语言中的具体实现。二者如同“蓝图”与“建筑”，是设计与施工的关系。



有限性 vs 无限性

算法必须在有限步骤后终止，而程序可以无限运行，如操作系统。

逻辑 vs 实现

算法强调 **解题逻辑**，程序侧重 **实现细节** 与特定语言语法。

算法的五大黄金准则



输入

零个或多个外部量



输出

至少产生一个量



确定性

指令清晰无歧义



有限性

步骤与时间都有限



有效性

基本到可用纸笔执行

算法可以用 自然语言、流程图、伪代码 等多种方式描述，核心是逻辑清晰、步骤可行。

系统化问题求解五步法

1

理解问题
抽象描述

2

选择结构
数据建模

3

设计算法
策略选择

4

实现程序
编码实现

5

分析结果
性能评估

以旅行商问题(TSP)为例，每一步都需做出关键决策，最终形成从问题到解决方案的完整闭环。

03

复杂度：度量算法效率的标尺



时间复杂度与空间复杂度

- 时间复杂度以 **基本操作执行次数** 刻画算法计算工作量
- 空间复杂度衡量算法运行所需的存储空间
- 两者共同构成资源占用模型，为硬件配置、功耗估算、性能优化提供量化依据



时间复杂度

执行算法所需的计算工作量



空间复杂度

算法执行过程中所需的存储空间

时间复杂性
和
空间复杂性

正确性

健壮性

复杂性

可读性

简单性

时间复杂度：相对度量

算法的时间复杂度是衡量其运行时间性能的 **相对度量**，它关注算法中简单操作（如赋值、比较）的执行次数，而非具体的机器时间。



这种度量方式 **独立于具体的机器和编译器**，使我们能客观地比较不同算法的效率。

举例：累加求和

```
int Sum ( int b[], int n )  
{  
    int s = 0;  
    for ( int i = 0; i < n; i ++ )  
        s += b[i];  
    return s;  
}
```

$$T(n) = 4n + 4$$

```
    int i = 0 //1次  
mark1: if( i >= n) goto mark 2; //n+1次  
    s += b[i]; //n次  
    i ++; //n次  
    goto mark1; //n次  
mark2: return s;
```



举例：简单选择排序

```
void SelectSort ( int b[], int n )  
{  
    int i, j, k, x;  
    for ( i=0; i <n-1; i ++)  
    {  
        k = i;  
        for ( j=i+1; j<n; j++)  
            if (b[k] > b[j]) k =j;  
        x = b[j];  
        b[i] = b[k];  
        b[k] = x;  
    }  
}
```



```

        i = 0; //1次
mark1: if(!(i < n-1)) goto mark4; //n次
        k =j ; //n-1次
        j=i+1; //n-1次
mark2: if (!j<n)) goto mark3; //(n+2)(n-1)/2次
        if (b[k] > b[j]) k =j; //n(n-1)/2次
        j++; //n(n-1)/2次
        goto mark2; //n(n-1)/2次

mark3: x = b[j]; //n-1次
        b[i] = b[k]; //n-1次
        b[k] = x; //n-1次
        i++; //n-1次
        goto mark1; //n-1次
mark4:

```

$$T(n) = 2n^2 + 7n - 7$$

[[Example]] Matrix addition

```

void add ( int a[ ][ MAX_SIZE ],
           int b[ ][ MAX_SIZE ],
           int c[ ][ MAX_SIZE ],
           int rows, int cols )
{
    int i, j ;
    for ( i = 0; i < rows; i++ )    /* rows + 1 */
        for ( j = 0; j < cols; j++ ) /* rows(cols+1) */
            c[ i ][ j ] = a[ i ][ j ] + b[ i ][ j ]; /* rows · cols */
}

```

A: Exchange
rows and cols.

$$T(\text{rows}, \text{cols}) = 2 \text{ rows} \cdot \text{cols} + 2\text{rows} + 1$$

[[Example]] Iterative function
for summing a list of numbers

$$T_{\text{sum}}(n) = 2n + 3$$

```
float sum ( float list[ ], int n )
{ /* add a list of numbers */
  float tempsum = 0; /* count = 1 */
  int i ;
  for ( i = 0; i < n; i++ )
    /* count ++ */
    tempsum += list [ i ]; /* count ++ */
  /* count ++ for last execution of for */
  return tempsum; /* count ++ */
}
```

[[Example]] Recursive function for
summing a list of numbers

$$T_{\text{rsum}}(n) = 2n + 2$$

But it takes more time to
compute each step.

```
float rsum ( float list[ ], int n )
{ /* add a list of numbers */
  if ( n ) /* count ++ */
    return rsum(list, n-1) + list[n - 1];
  /* count ++ */
  return 0; /* count ++ */
}
```

§ What to Analyze

最坏、最好与平均情形

同一算法对不同输入实例表现差异巨大。三者并存，为算法选型与风险评估提供全景视角。



最坏情况

给出性能上界，用于承诺系统能力。



最好情况

揭示优化潜力，指明性能上限。



平均情况

反映日常使用体验，依赖输入分布。

04

渐近符号五重奏



大O符号 (Big O Notation)

大O符号描述了函数在输入趋向无穷大时的渐近上界，用于表示算法的最坏情况运行时间。

数学定义

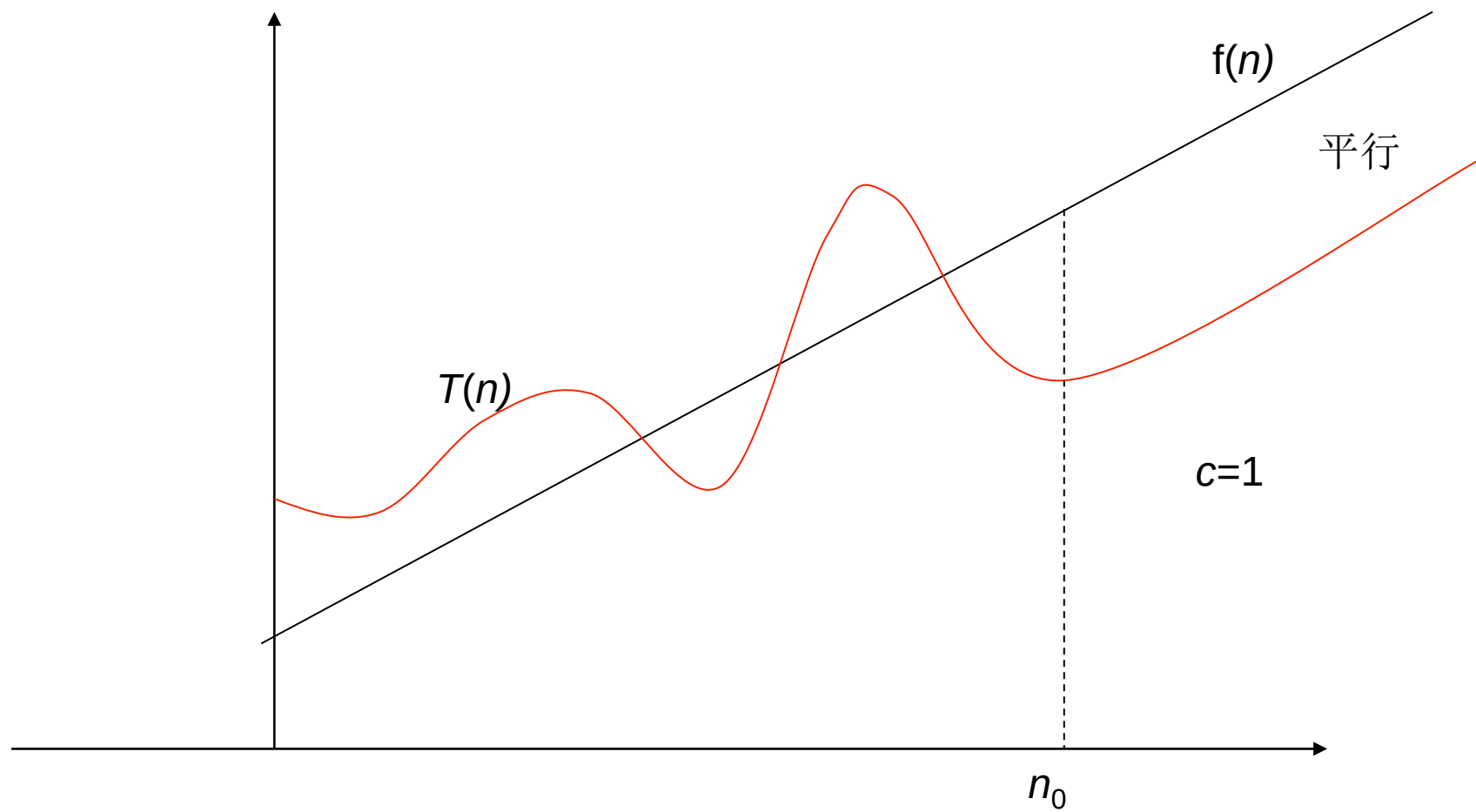
若存在正常数 c 和 n_0 ，使得对于所有的 $n \geq n_0$ ，都有：

$$T(n) \leq c \cdot f(n)$$

则称 $T(n) = O(f(n))$ 。

直观理解：当 n 足够大时， $T(n)$ 的增长速度不超过 $f(n)$ 的某个常数倍。

渐近上界记号 O



大Ω符号 (Big Omega Notation)

大Ω符号描述了函数的渐近下界，用于表示算法的最好情况运行时间或某个问题的计算复杂度下界。

数学定义

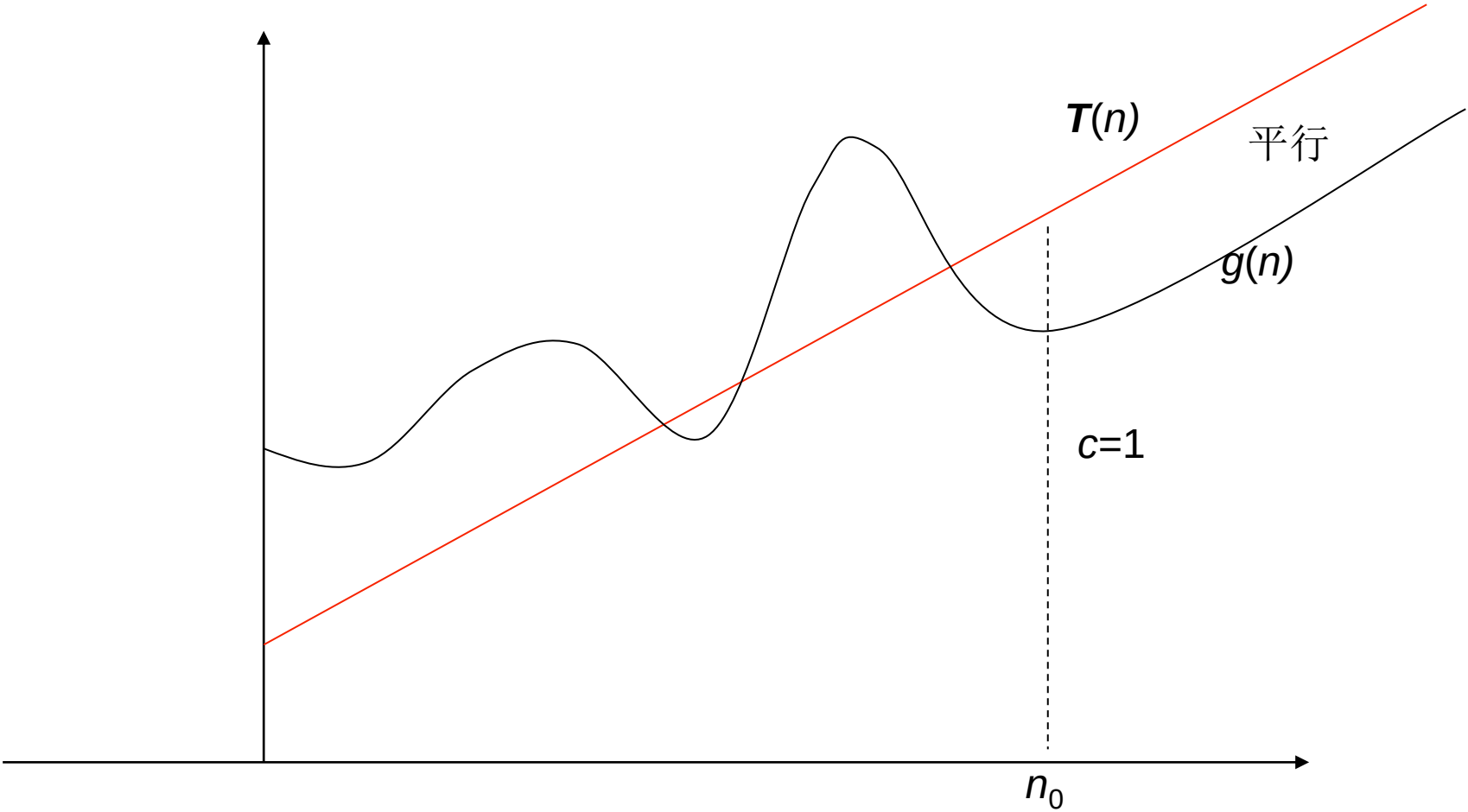
若存在正常数 c 和 n_0 ，使得对于所有的 $n \geq n_0$ ，都有：

$$T(n) \geq c \cdot g(n)$$

则称 $T(n) = \Omega(g(n))$ 。

直观理解：当 n 足够大时， $T(n)$ 的增长速度不低于 $g(n)$ 的某个常数倍。

渐近下界记号 Ω



大Θ符号 (Big Theta Notation)

大Θ符号描述了函数的 **紧确界**，即同时满足大O和大Ω条件，表示算法的运行时间与某函数同阶。

数学定义

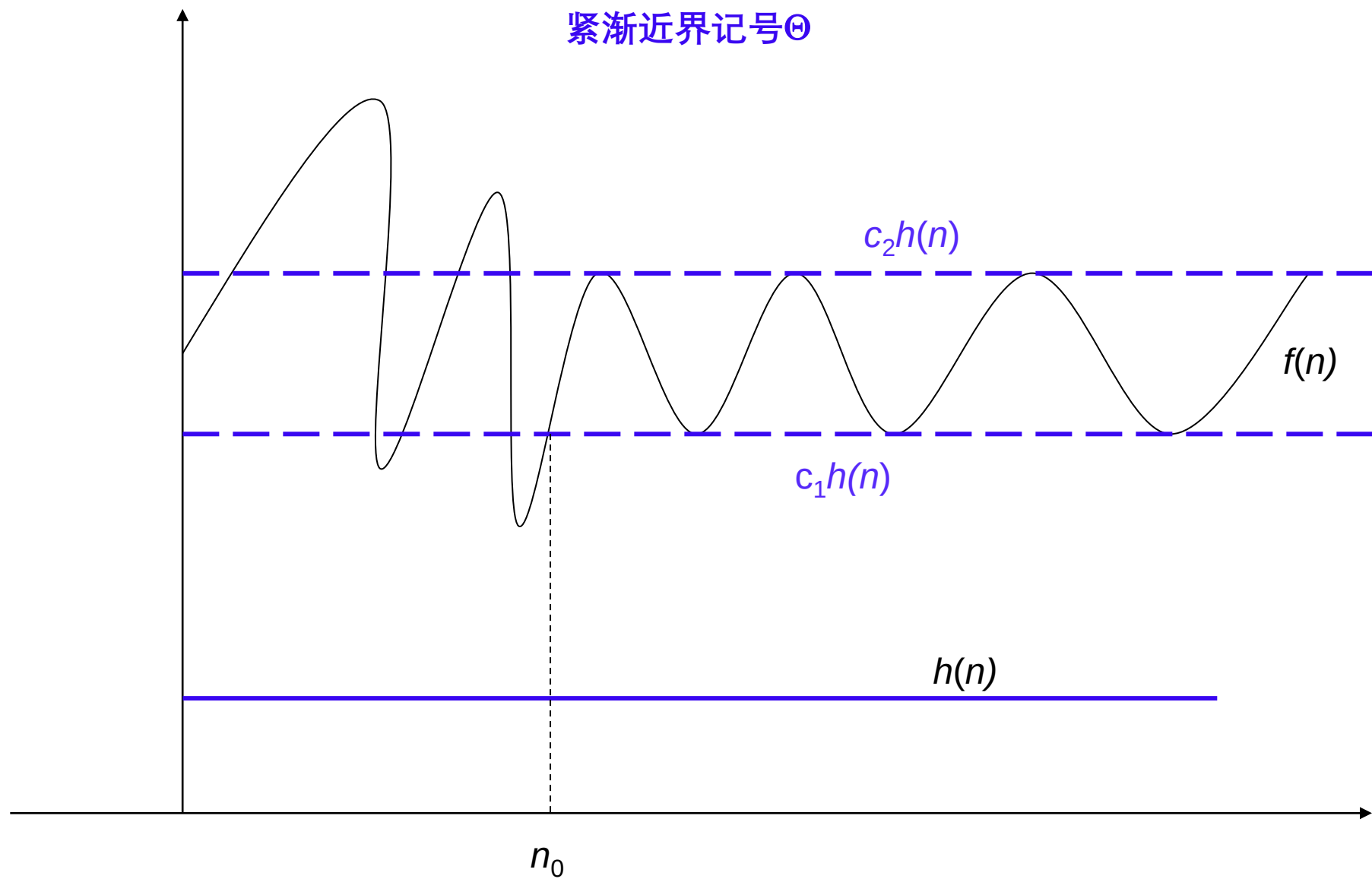
若存在正常数 c_1, c_2 和 n_0 ，使得对于所有的 $n \geq n_0$ ，都有：

$$c_1 \cdot h(n) \leq T(n) \leq c_2 \cdot h(n)$$

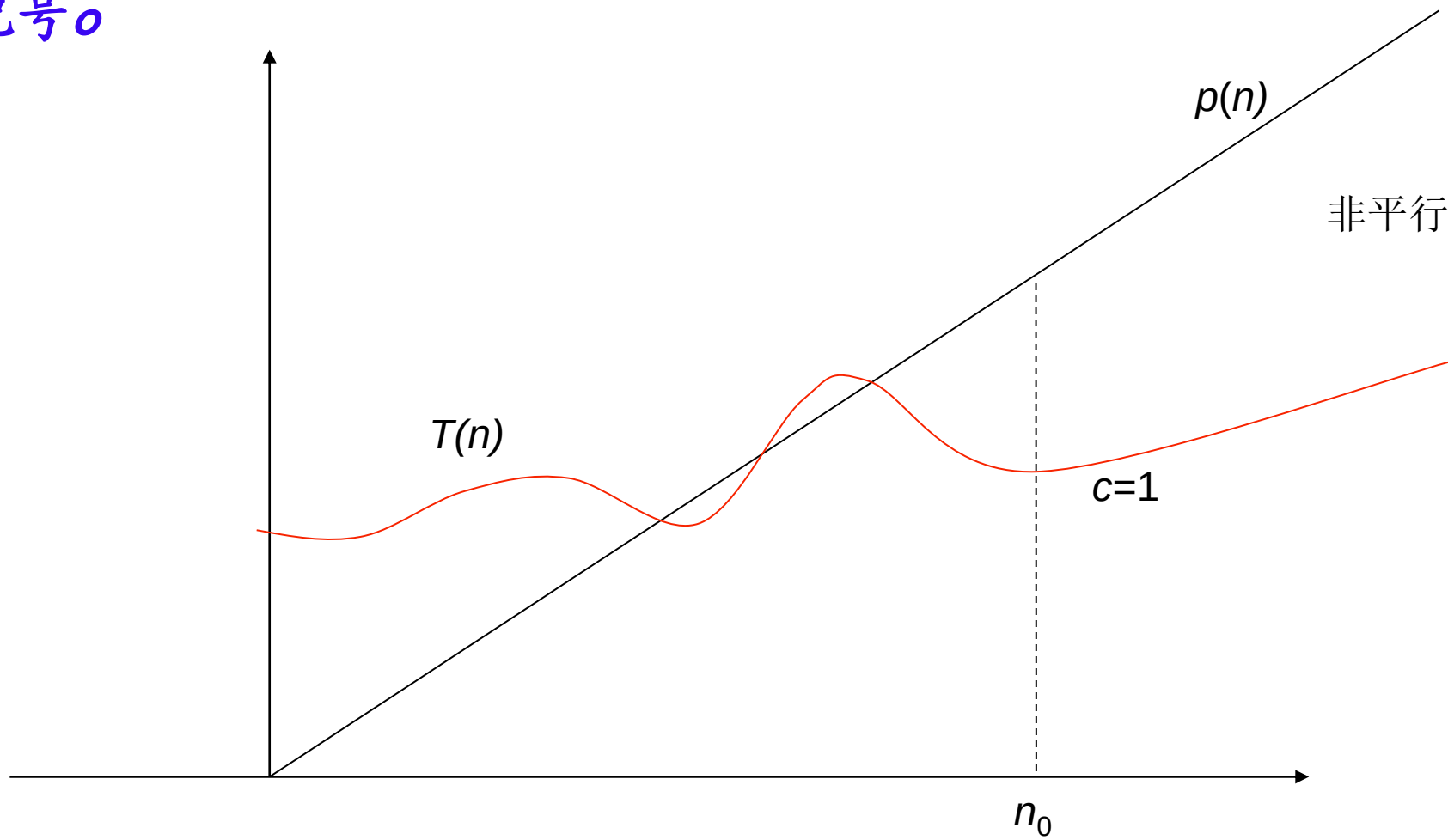
则称 $T(n) = \Theta(h(n))$ 。

直观理解：当 n 足够大时， $T(n)$ 的增长速度与 $h(n)$ **同阶**。

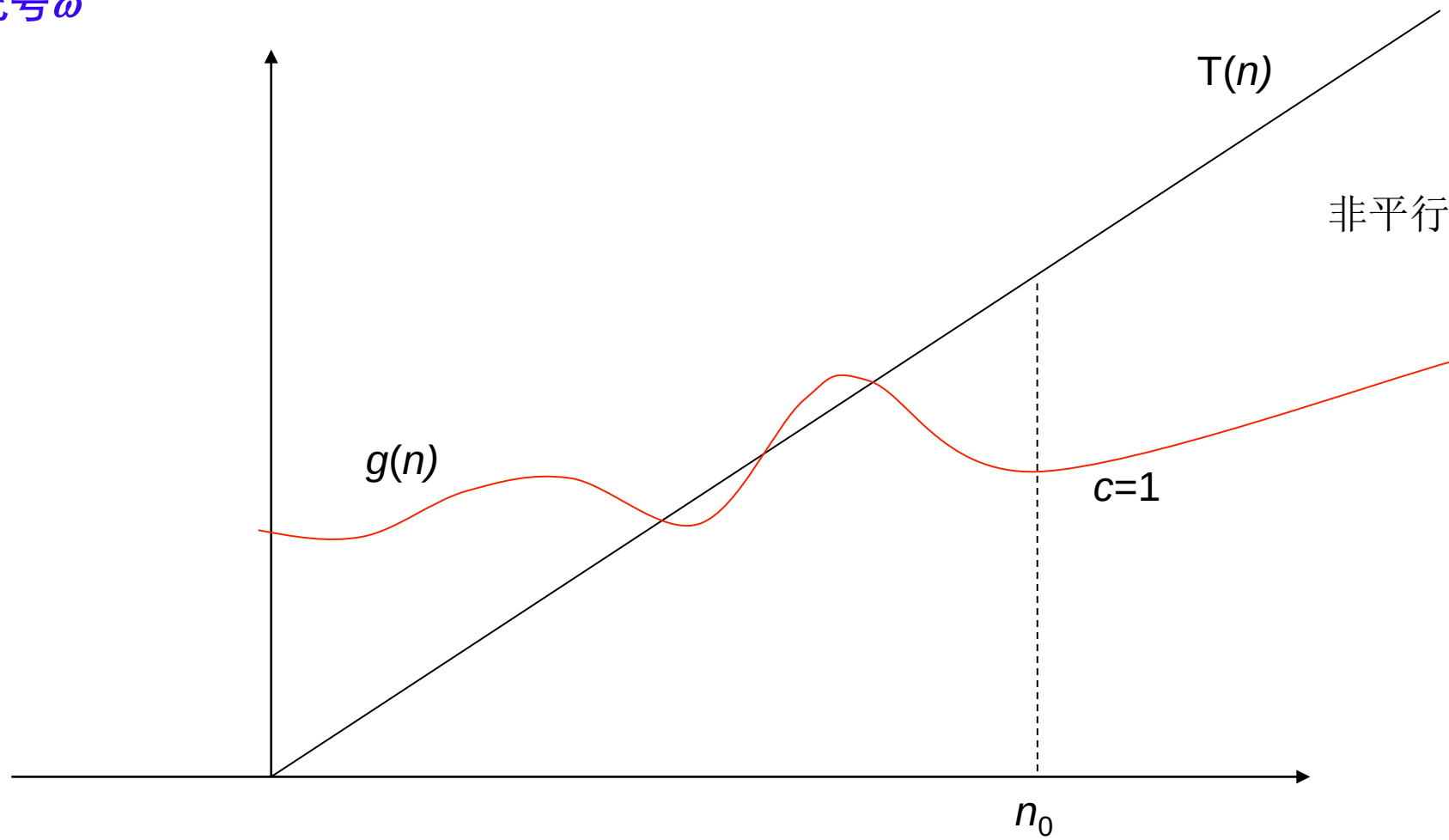
紧渐近界记号 Θ



非紧上界记号 $\mathcal{O}$



非紧下界记号 ω



大O、Ω、Θ：渐近记号体系

记号体系忽略常数与低阶项，使不同算法在输入规模趋于无穷时可横向比较，为理论计算机科学提供通用“度量衡”。

O (Big-O)

渐近上界

$f(n)$ 的增长速度不超过 $g(n)$ 的某个常数倍。

Ω (Big-Omega)

渐近下界

$f(n)$ 的增长速度至少是 $g(n)$ 的某个常数倍。

Θ (Big-Theta)

紧渐近界

$f(n)$ 的增长速度与 $g(n)$ 在常数因子范围内相同。

Note:

➤ $2N + 3 = O(N) = O(N^{k \geq 1}) = O(2^N) = \dots$ We shall always take the **smallest** $f(N)$.

➤ $2^N + N^2 = \Omega(2^N) = \Omega(N^2) = \Omega(N) = \Omega(1) = \dots$ We shall always take the **largest** $g(N)$.

小o与ω：严格意义上的快慢

小o符号 (Little o)
表示 严格小于 的关系

。

$$T(n) = o(p(n))$$

即

$$\lim_{n \rightarrow \infty} T(n)/p(n) = 0$$

例: $n \log n = o(n^2)$

小ω符号 (Little ω)
表示 严格大于 的关系

。

$$T(n) = \omega(q(n))$$

即 $\lim_{n \rightarrow \infty} T(n)/q(n) = \infty$

$$\lim_{n \rightarrow \infty} T(n)/q(n) = \infty$$

例: $n^2 = \omega(n \log n)$

渐近运算的封闭法则

这些法则允许我们像处理代数表达式一样处理渐近符号，简化复杂度分析。

$$O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$$

$$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$$

$$O(c \cdot f(n)) = O(f(n))$$

$$\text{若 } g(n)=O(f(n)) \quad \text{则} \quad O(f(n) + g(n)) = O(f(n))$$

渐近符号的性质

理解这些性质有助于我们进行更复杂的复杂度推导和比较。



传递性

若 $f = O(g)$ 且 $g = O(h)$, 则 $f = O(h)$ 。



自反性

$f = O(f)$ 。



对称性

$f = \Theta(g)$ 当且仅当 $g = \Theta(f)$ 。



互对称性

$f = O(g)$ 当且仅当 $g = \Omega(f)$ 。



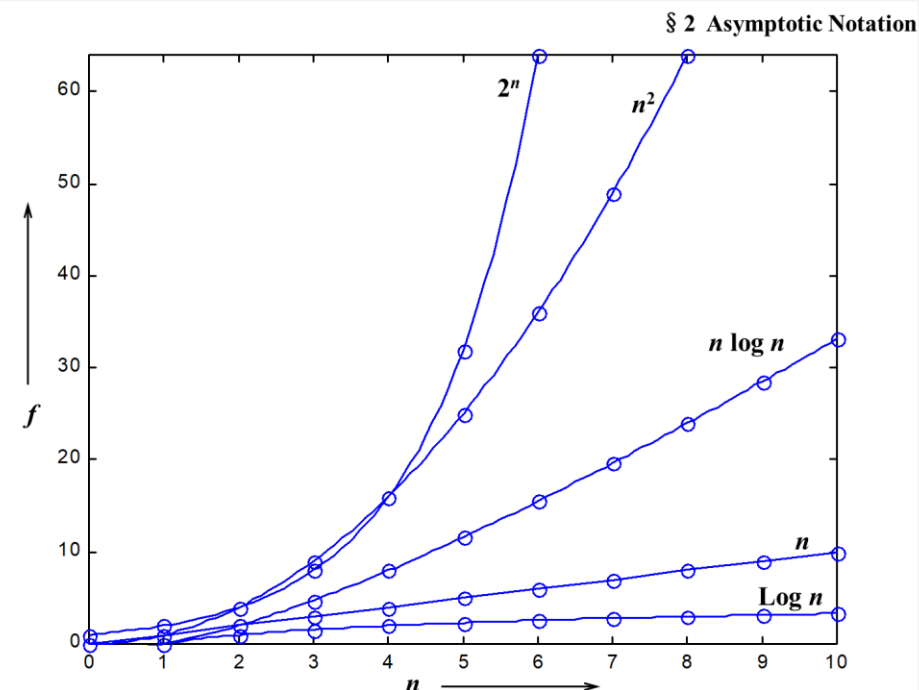
类比关系

O, Ω, Θ 可类比于 $\leq, \geq, =$ 。

常见复杂度阶与典型算法

时间复杂度	名称	典型算法示例
$O(1)$	常数时间	访问数组中特定索引的元素
$O(\log n)$	对数时间	二分查找算法
$O(n)$	线性时间	遍历数组或链表
$O(n \log n)$	线性对数时间	归并排序、快速排序（平均情况）
$O(n^2)$	平方时间	冒泡排序、插入排序、选择排序
$O(2^n)$	指数时间	汉诺塔问题的递归解法、背包问题的暴力解法

Input size n							
Time	Name	1	2	4	8	16	32
1	constant	1	1	1	1	1	1
$\log n$	logarithmic	0	1	2	3	4	5
n	linear	1	2	4	8	16	32
$n \log n$	log linear	0	2	8	24	64	160
n^2	quadratic	1	4	16	64	256	1024
n^3	cubic	1	8	64	512	4096	32768
2^n	exponential	2	4	16	256	65536	4294967296
$n!$	factorial	1	2	24	40326	2092278988000	26313×10^{33}



§ 2 Asymptotic Notation

	Time for $f(n)$ instructions on a 10^9 instr/sec computer						
n	$f(n)=n$	$n \log_2 n$	n^2	n^3	n^4	n^{10}	2^n
10	.01 μ s	.03 μ s	.1 μ s	1 μ s	10 μ s	10sec	1 μ s
20	.02 μ s	.09 μ s	.4 μ s	8 μ s	160 μ s	2.84hr	1ms
30	.03 μ s	.15 μ s	.9 μ s	27 μ s	810 μ s	6.83d	1sec
40	.04 μ s	.21 μ s	1.6 μ s	64 μ s	2.56ms	121.36d	18.3min
50	.05 μ s	.28 μ s	2.5 μ s	125 μ s	6.25ms	3.1yr	13d
100	.10 μ s	.66 μ s	10 μ s	1ms	100ms	3171yr	$4 \cdot 10^{13}$ yr
1,000	1.00 μ s	9.96 μ s	1ms	1sec	16.67min	$3.17 \cdot 10^{13}$ yr	$32 \cdot 10^{283}$ yr
10,000	10 μ s	130.03 μ s	100ms	16.67min	115.7d	$3.17 \cdot 10^{23}$ yr	
100,000	100 μ s	1.66ms	10sec	11.57d	3171yr	$3.17 \cdot 10^{33}$ yr	
1,000,000	1.0ms	19.92ms	16.67min	31.71yr	$3.17 \cdot 10^7$ yr	$3.17 \cdot 10^{43}$ yr	

μ s = microsecond = 10^{-6} seconds

ms = millisecond = 10^{-3} seconds

sec = seconds

min = minutes

hr = hours

d = days

yr = years

05

复杂度估算实战与主定理



非递归分析：以冒泡排序为例

通过统计循环与比较操作的执行次数，可得 **最坏情形约 $\frac{1}{2}n^2$ 次比较**，复杂度 $O(n^2)$ ；最好情形数组已有序，仅需线性扫描一次，复杂度 $O(n)$ 。常数级辅助变量使空间复杂度为 $O(1)$ 。

冒泡排序伪代码

```
for i = 1 to n-1
  swapped = false
  for j = 1 to n-i
    if arr[j] > arr[j+1]
      swap(arr[j], arr[j+1])
      swapped = true
  if not swapped break
```

时间复杂性渐进阶分析的规则(最坏情况)

- 1). 赋值,比较,算术运算,逻辑运算,读写单个变量(常量)只需1单位时间
- 2). 执行条件语句 **if c then S1 else S2** 的时间为 $T_C + \max(T_{S1}, T_{S2})$.
- 3). 选择语句 **case A of a1: s1; a2: s2; ...; am: sm**
需要的时间为 $\max(T_{S1}, T_{S2}, \dots, T_{Sm})$.
- 4). 访问数组的单个分量或纪录的单个域需要一个单位时间.
- 5). 执行**for**循环语句的时间=执行循环体时间*循环次数.
- 6). **while c do s (repeat s until c)**语句时间= $(T_C + T_S)$ *循环次数.
- 7). 用**goto**从循环体内跳到循环体末或循环后面的语句时,不需额外时间
- 8). 过程或函数调用语句
对非递归调用,根据调用层次由里向外用规则1-7进行分析;
对递归调用,可建立关于 **$T(n)$** 的递归方程,求解该方程得到 **$T(n)$** .

递归求解技法



递归方程解的渐近阶的求法

1. **代入法** 这个方法的基本步骤是先推测递归方程的显式解，然后用数学归纳法证明这一推测的正确性。那么，显式解的渐近阶即为所求。
2. **迭代法** 这个方法的基本步骤是通过反复迭代，将递归方程的右端变换成一个级数，然后求级数的和，再估计和的渐近阶；或者，不求级数的和而直接估计级数的渐近阶，从而达到对递归方程解的渐近阶的估计。
3. **套用公式法** 这个方法针对形如： $T(n) = aT(n/b) + f(n)$ 的递归方程，给出三种情况下方程解的渐近阶的三个相应估计公式供套用。
4. **差分方程法** 有些递归方程可以看成一个差分方程，因而可以用解差分方程(初值问题)的方法来解递归方程。然后对得到的解作渐近阶的估计。
5. **母函数法** 这是一个有广泛适用性的方法。它不仅可以用来求解线性常系数高阶齐次和非齐次的递归方程，而且可以用来求解线性变系数高阶齐次和非齐次的递归方程，甚至可以用来求解非线性递归方程。方法的基本思想是设定递归方程解的母函数，努力建立一个关于母函数的可解方程，将其解出，然后返回递归方程的解。



求解递归式：代入法与迭代法

当算法用递归定义时，其复杂度通常表示为递归式。求解递归式是分析递归算法性能的关键。

💡 代入法

1. **猜测** 解的形式。
2. 用 **数学归纳法** 证明猜测的正确性。

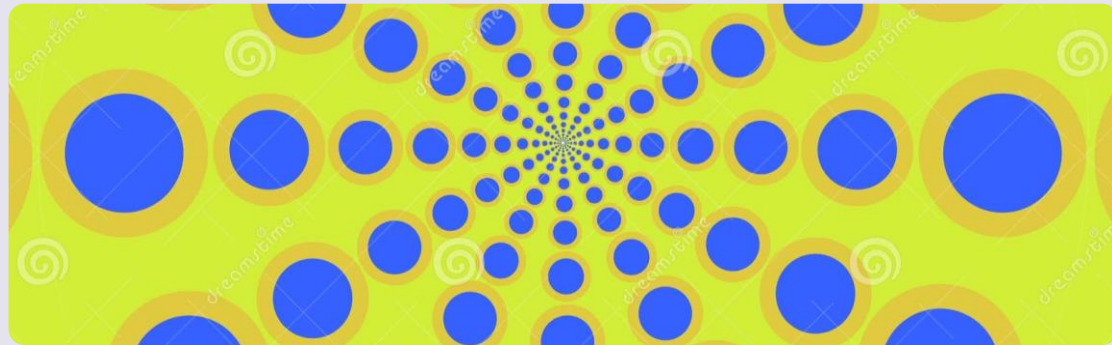
适用于能根据经验猜出答案的情况。



🔄 迭代法

1. **层层展开** 递归式，将其表示为求和形式。
2. 利用 **级数求和** 公式计算结果。

直观但计算可能复杂，适合展开后模式清晰的递归式。



代入法 猜测技术:对递推关系式估计一个上限,
然后(用数学归纳法)证明它正确。

$$T(n) = 2T(n/2) + n \quad \text{证明} \quad T(n) = O(n \log n)$$

- 推测 $T(n) = O(n \log n)$, 即推测存在正的常数 C 和自然数 n_0 , 使得当 $n \geq n_0$ 时有

$$T(n) \leq Cn \log n \quad (1-1)$$

- 假设假设当 $2^{k-1}n_0 \leq n \leq 2^k n_0$, $k \geq 1$ 时,
(1-1) 成立。

- 那么, 当 $2^k n_0 \leq n \leq 2^{k+1} n_0$ 时, 有:
$$\begin{aligned} T(n) &= 2T(\lfloor n/2 \rfloor) + n \\ &\leq 2C \lfloor n/2 \rfloor \cdot \log(\lfloor n/2 \rfloor) + n \\ &< 2C \cdot \frac{n}{2} \cdot \log \frac{n}{2} + n \\ &= Cn \log n - Cn + n \\ &= Cn \log n - (C-1)n \\ &\leq Cn \log n \end{aligned}$$

算法复杂性分析中常见函数

- 这个方法的局限性在于它只适合容易推测出答案的递归方程或善于进行推测的高手。推测递归方程的正确解，没有一般的方法，得靠经验的积累和洞察力。

FUNCTION	NAME
c	Constant
$\log N$	Logarithmic
$\log^2 N$	Log-squared
N	Linear
$N \log N$	$N \log N$
N^2	Quadratic
N^3	Cubic
2^N	Exponential

迭代法

方法的思想是迭代地展开递归方程的右端,使之成为一个非递归的和式,然后通过对和式的估计来达到对方程左端即方程的解的估计。

$$T(n) = 3T(n/4) + n$$

$$T(n) \leq n + 3(n/4 + 3(n/4^2 + 3(n/4^3 + \dots + 3(n/4^i + 3T(0)) \dots)))$$

$$= n + \frac{3}{4}n + \frac{3^2}{4^2}n + \frac{3^3}{4^3}n + \dots + \frac{3^i}{4^i}n + 3^{i+1} \cdot T(0)$$

$$< 4n + 3^{i+1} \cdot T(0)$$

知 $i \leq \log_4 n$, 从而

$$3^{i+1} \leq 3^{\log_4 n + 1} = 3^{\log_3 n \cdot \log_4 3 + 1} = 3n^{\log_4 3} ,$$

代入

$$T(n) < 4n + 3n^{\log_4 3} \cdot T(0)$$

- 即, $T(n) = O(n)$

$$T(n) = \begin{cases} 7 & n = 1 \\ 2T(n/2) + 5n^2 & n > 1 \end{cases}$$

$$T(n) = 2T(n/2) + 5n^2$$

$$= 2(2T(n/4) + 5(n/2)^2) + 5n^2$$

$$= 2(2(2T(n/8) + 5(n/4)^2) + 5(n/2)^2) + 5n^2$$

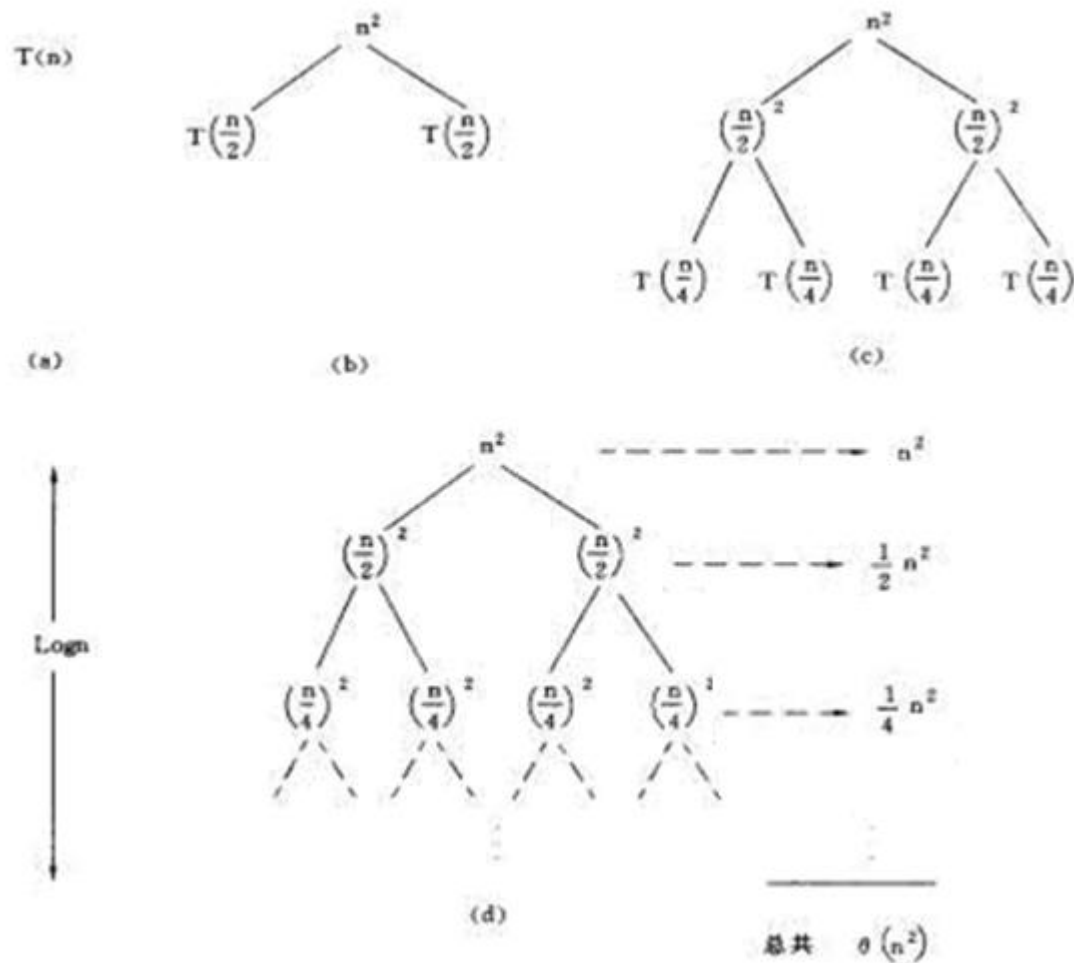
$$= 2^k T(1) + 2^{k-1} 5 \left(\frac{n}{2^{k-1}}\right)^2 + \dots + 2 \times 5 \left(\frac{n}{2}\right)^2 + 5n^2$$

$$T(n) = 7n + 5 \sum_{i=0}^{k-1} \left(\frac{n}{2^i}\right)^2 = 7n + 5n^2 \left(2 - \frac{1}{2^{k-1}}\right) = 10n^2 - 3n \leq 10n^2 = O(n^2)$$



$$T(n) = 2T(n/2) + n^2$$

表格化的办法：先按横向求出每层结点的值之和，并记录在各相应层右端顶格处，然后从根到叶逐层地将顶格处的结果加起来便是我们要求的结果。照此，我们得到解的渐近阶为 $\theta(n^2)$ 。



$$T(n) = T(n/3) + T(2n/3) + n$$

当我们累计递归树各层的值时，得到每一层的和都等于 n ，从根到叶的最长路径是

$$n \rightarrow \frac{2}{3}n \rightarrow \left(\frac{2}{3}\right)^2 n \rightarrow \dots \rightarrow 1$$

设最长路径的长度为 k ，则应该有

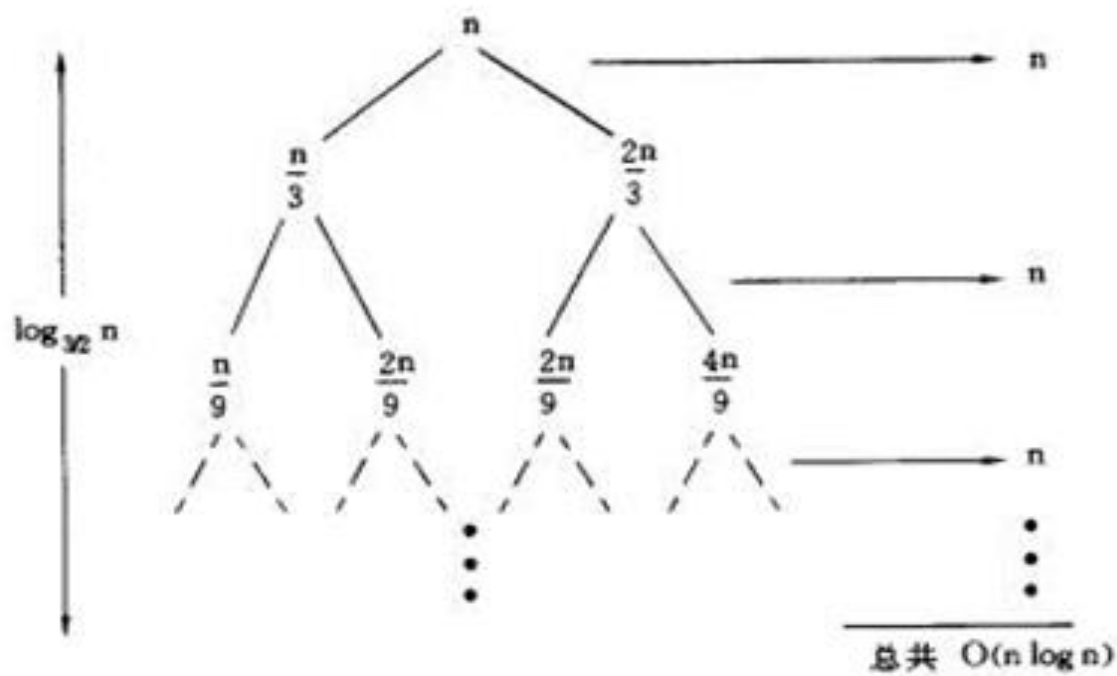
$$\left(\frac{2}{3}\right)^k n = 1,$$

得

$$k = \log_{3/2} n,$$

于是

$$T(n) \leq \sum_{i=0}^k n = (k+1)n = n(\log_{3/2} n + 1)$$



求解递归式：主定理 (Master Theorem)

主定理是求解形如 $T(n) = aT(n/b) + f(n)$ 的分治递归式的“万能钥匙”。

标准形式

$$T(n) = aT(n/b) + \Theta(n^k)$$

其中 $a \geq 1, b > 1, k \geq 0$ 为常数。

三种情况

1. 若 $a > b^k$, 则 $T(n) = \Theta(n^{\log_b a})$
2. 若 $a = b^k$, 则 $T(n) = \Theta(n^k \log n)$
3. 若 $a < b^k$, 则 $T(n) = \Theta(n^k)$

套用公式法

大小为 n 的原问题分成若干个大小为 n/b 的子问题，其中 a 个子问题需要求解，而 cn^k 是合并各个子问题的解需要的工作量。

$$T(n) = \begin{cases} c & n = 1 \\ aT(n/b) + cn^k & n > 1 \end{cases}$$

$$T(n) = \begin{cases} O(n^{\log_b a}) & a > b^k \\ O(n^k \log_b n) & a = b^k \\ O(n^k) & a < b^k \end{cases}$$

套用公式法

若递归方程形如: $T(n)=aT(n/b)+f(n)$, 可根据 $f(n)$ 的不同情况套用公式

- 1) 若 $\exists \varepsilon > 0$, 使得 $f(n) = O(n^{\log_b a - \varepsilon})$, 则 $T(n) = \theta(n^{\log_b a})$
- 2) 若 $f(n) = \theta(n^{\log_b a})$, 则 $T(n) = \theta(n^{\log_b a} \cdot \log n)$
- 3) 若 $\exists \varepsilon > 0$, 使得 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 且 $\exists c < 1$, 当 n 充分大时有 $a f(n/b) \leq c f(n)$, 则 $T(n) = \theta(f(n))$



母函数法

母函数法：设 $a_0, a_1, \dots, a_n, \dots$ 是任意数列，称 $f(x) = a_0 + a_1x + \dots + a_nx^n + \dots$ 为数列的母函数。若取数列为算法的时间复杂性函数 $\{T(n)\}$ ，则其母函数为： $f(x) = T(0) + T(1)x + \dots + T(n)x^n + \dots$

若能由 $T(n)$ 的递归方程求出 $T(n)$ 的母函数，其展开式第 n 项系数即为 $T(n)$ 。



例1 考虑线性变系数二阶齐次递归方程

$$(n-1)T(n)=(n-2)T(n-1)+2T(n-2), \quad n \geq 2$$

和初始条件 $T(0)=0, T(1)=1$ 。根据初始条件，可计算 $T(2)=0, T(3)=T(1)=1$ 。

设 $\{T(n)\}$ 的母函数为：

$$A(x) = \sum_{n=0}^{\infty} T(n)x^n$$

由于 $T(0)=T(2)=0, T(1)=1$ ，有：

$$\begin{aligned} A(x) &= x + T(3)x^3 + T(4)x^4 + \cdots + T(n)x^n + \cdots \\ &= x(1 + T(3)x^2 + T(4)x^3 + \cdots + T(n)x^{n-1} + \cdots) \end{aligned}$$

令 $B(x) = A(x)/x$ ，即：

$$B(x) = 1 + T(3)x^2 + T(4)x^3 + \cdots + T(n)x^{n-1} + \cdots$$

$$B(x) = 1 + T(3)x^2 + T(4)x^3 + \cdots + T(n)x^{n-1} + \cdots$$

那么：

$$B'(x) = 2T(3)x + 3T(4)x^2 + \cdots + (n-1)T(n)x^{n-2} + \cdots$$

$$xB'(x) = 2T(3)x^2 + 3T(4)x^3 + \cdots + (n-1)T(n)x^{n-1} + \cdots$$

$$B'(x) - xB'(x) = 2T(3)x + (3T(4) - 2T(3))x^2 + (4T(5) - 3T(4))x^3 + \cdots + ((n-1)T(n) - (n-2)T(n-1))x^{n-2} + \cdots$$

代入 $T(3) = 1$ ，得

$$\begin{aligned} B'(x)(1-x) &= 2T(1)x + 2T(2)x^2 + 2T(3)x^3 + \cdots + 2T(n-2)x^{n-2} + \cdots \\ &= 2x(1 + T(3)x^2 + T(4)x^3 + \cdots + T(n)x^{n-1} + \cdots) \\ &= 2xB(x) \end{aligned}$$

即

$$\frac{B'(x)}{B(x)} = \frac{2x}{1-x} = -2 + \frac{2}{1-x}$$

两边同时沿 $[0, x]$ 积分，并注意到 $B(0)=1$ ，有：

$$\ln B(x) = -2x - 2\ln(1-x)$$

$$B(x) = e^{-2x} / (1-x)^2$$

把 $B(x)$ 展开成幂级数，得

$$\begin{aligned} B(x) &= \sum_{n=0}^{\infty} \frac{(-2x)^n}{n!} \sum_{n=0}^{\infty} \binom{n+1}{1} x^n \\ &= \sum_{i=0}^{\infty} \frac{(-1)^i \cdot 2^i}{i!} x^i \cdot \sum_{j=0}^{\infty} (j+1) \cdot x^j \\ &= \sum_{i=0}^{\infty} \sum_{j=0}^{\infty} \frac{(-1)^i \cdot 2^i \cdot (j+1)}{i!} x^{(i+j)} \\ &= \sum_{n=0}^{\infty} \left(\sum_{i=0}^n \frac{(-1)^i \cdot 2^i \cdot (n-i+1)}{i!} \right) \cdot x^n \end{aligned} \quad 0 \leq x < 1$$

从而

$$\begin{aligned}
 A(x) &= \sum_{n=0}^{\infty} \left(\sum_{i=0}^n \frac{(-1)^i \cdot 2^i \cdot (n-i+1)}{i!} \right) \cdot x^{n+1} \\
 &= \sum_{n=1}^{\infty} \left(\sum_{i=0}^{n-1} \frac{(-1)^i \cdot 2^i \cdot (n-i)}{i!} \right) \cdot x^n \qquad 0 \leq x < 1
 \end{aligned}$$

最后得

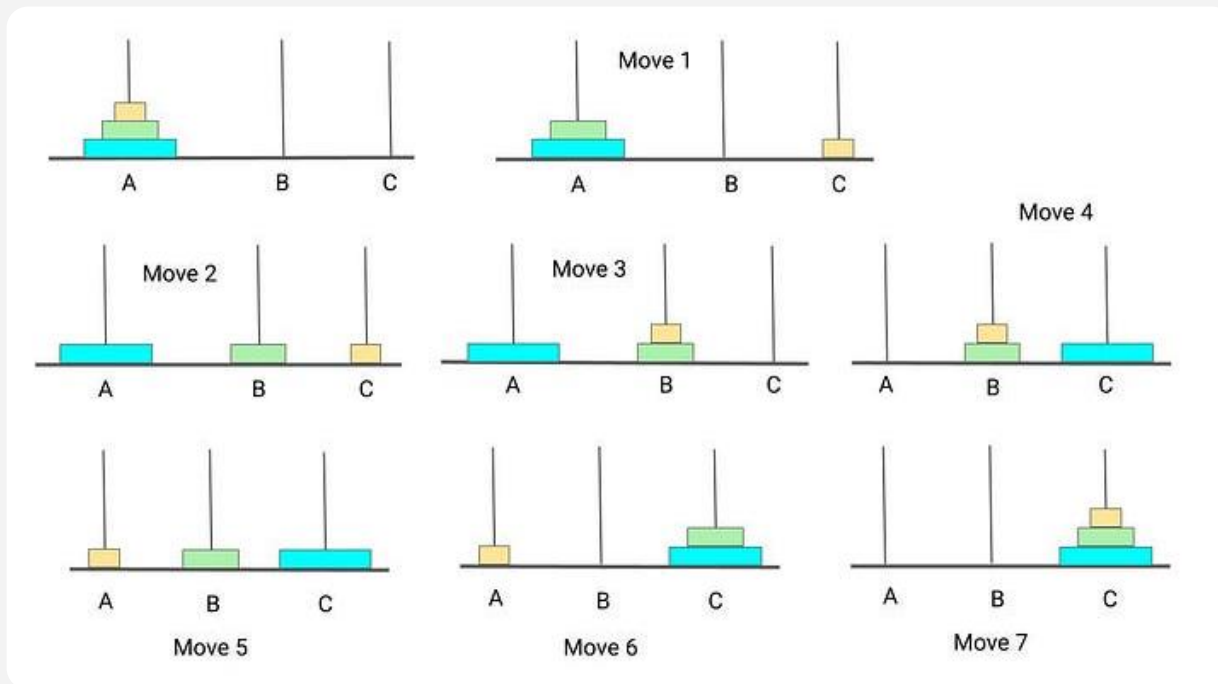
$$T(n) = \sum_{i=1}^{n-1} \frac{(-1)^i \cdot 2^i \cdot (n-i)}{i!} \qquad n \geq 1$$

递归分析：汉诺塔移动次数

设 $T(n)$ 为移动 n 个圆盘所需步数，可得递推式：

$$T(n) = 2T(n - 1) + 1$$

解得： $T(n) = 2^n - 1$



故 时间复杂度 $O(2^n)$ ；递归深度为 n ，空间复杂度 $O(n)$ 。该例展示如何用递推关系刻画指数级增长。

主定理：分治算法的万能钥匙

对形如 $T(n)=aT(n/b)+f(n)$ 的递推，主定理依 $f(n)$ 与 $n^{\log_b a}$ 的渐近关系分三情形给出 Θ 界。

归并排序 $a=b=2, f(n)=\Theta(n)$ 落入情形二，直接得 $\Theta(n \log n)$ 。
主定理省去展开递推的繁琐，成为分治复杂度分析的常备工具。

主定理公式

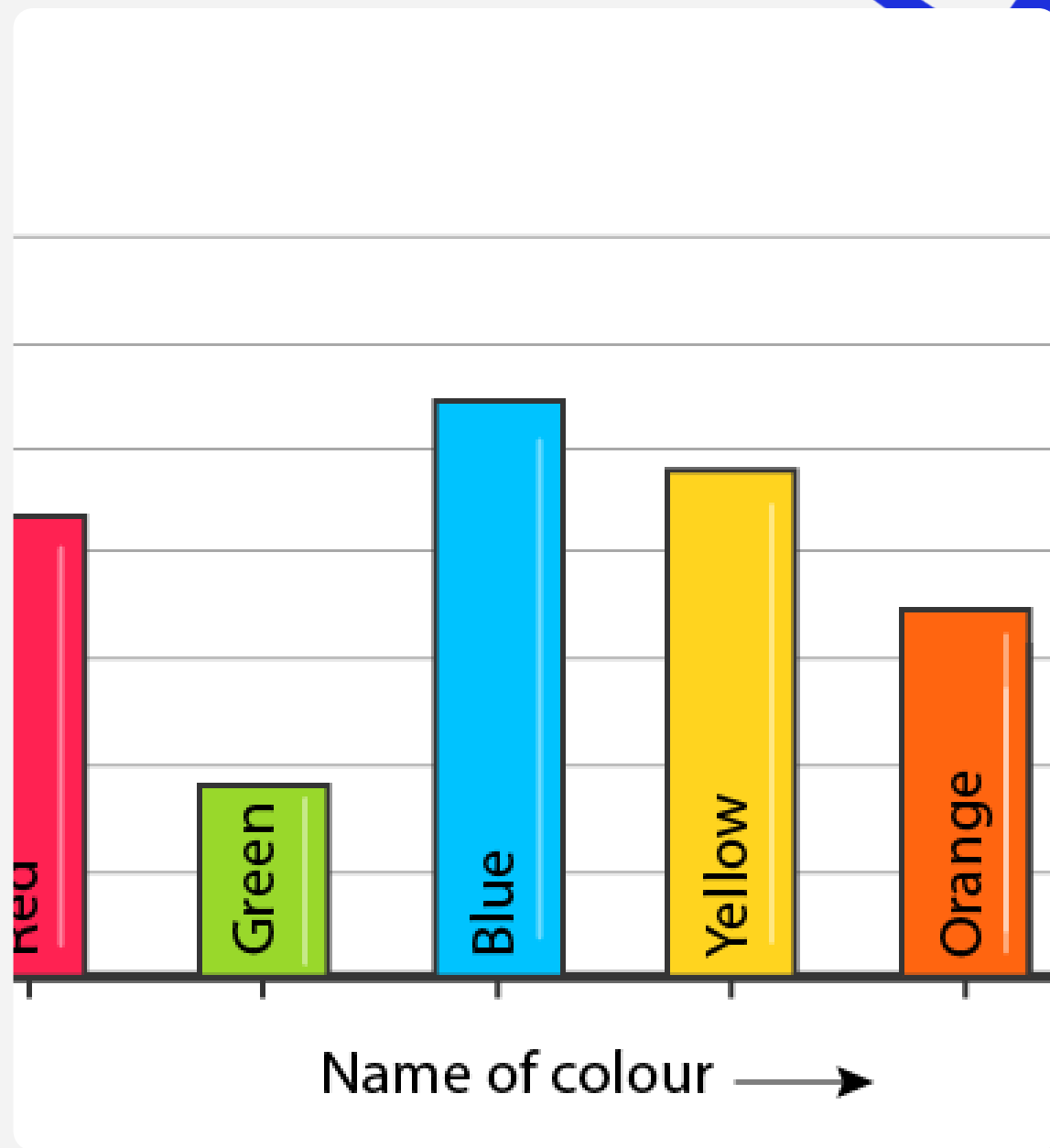
$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

若 $f(n)=\Theta(n \log_b a)$

则 $T(n)=\Theta(n \log_b a \log n)$

数值拟合：实验估阶的工程法

当理论分析困难时，可在多规模输入下实测运行时间，再用 n 、 $\log n$ 、 n^2 等模板函数拟合曲线。通过比较拟合优度反推渐近阶。此法虽不如理论严格，但能快速指导工程优化，并与分析结果相互验证。



复杂度实战指南



§ 3 Compare the Algorithms

【Example】 Given (possibly negative) integers $A_1, A_2, \dots, A_N$, find the maximum value of

$$\sum_{k=i}^j A_k.$$

给定由 n 个整数(可能为负整数)组成的序列 $a_1, a_2, \dots, a_n$, 求该序列形如 $\sum_{k=i}^j a_k$ 的子段和的最大值。当所有整数均为负整数时定义其最大子段和为 0。依此定义, 所求的最优值为:

$$\max \left\{ 0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k \right\}$$

例如当 $(a_1, a_2, a_3, a_4, a_5, a_6) = (-2, 11, -4, 13, -5, -2)$ 时, 最大子段和为 $\sum_{k=2}^4 a_k = 20$ 。

1,-5, 3,4,-2,-3,10

以 **1** 开始的子数组:

1↓

1,-5↓

1,-5,3↓

1,-5,3,4↓

1,-5,3,4,-2↓

1,-5,3,4,-2,-3↓

1,-5,3,4,-2,-3,10↓

以 **-5** 开始的子数组:

-5↓

-5,3↓

-5,3,4↓

-5,3,4,-2↓

-5,3,4,-2,-3↓

-5,3,4,-2,-3,10↓

以 **3** 开始的子数组: 略...↓

以 **4** 开始的子数组: 略...↓

以 **-2** 开始的子数组: 略...↓

以 **-3** 开始的子数组: 略...↓

以 **10** 开始的子数组: 略...↓

枚举后再对所有的子数组求和，可以得到如下代码

§ Compare the Algorithms

[[Example]] Given (possibly negative) integers $A_1, A_2, \dots, A_N$, find the maximum value of

$$\sum_{k=i}^j A_k.$$

Algorithm 1

```
int MaxSubsequenceSum ( const int A[ ], int N )
{
    int ThisSum, MaxSum, i, j, k;
    /* 1*/ MaxSum = 0; /* initialize the maximum sum */
    /* 2*/ for( i = 0; i < N; i++ ) /* start from A[ i ] */
    /* 3*/     for( j = i; j < N; j++ ) { /* end at A[ j ] */
    /* 4*/         ThisSum = 0;
    /* 5*/         for( k = i; k <= j; k++ )
    /* 6*/             ThisSum += A[ k ]; /* sum of A[ i ] to A[ j ] */
    /* 7*/         if ( ThisSum > MaxSum )
    /* 8*/             MaxSum = ThisSum; /* update max sum */
    /* 9*/     } /* end for-j and for-i */
    return MaxSum;
}
```

Detailed analysis
is your first
homework.

$$T(N) = O(N^3)$$

Algorithm 1

```
int MaxSubsequenceSum ( const int A[ ], int N )
{
    int ThisSum, MaxSum, i, j, k;
    /* 1*/ MaxSum = 0; /* initialize the maximum sum */
    /* 2*/ for( i = 0; i < N; i++ ) /* start from A[ i ] */
    /* 3*/     for( j = i; j < N; j++ ) { /* end at A[ j ] */
    /* 4*/         ThisSum = 0;
    /* 5*/         for( k = i; k <= j; k++ )
    /* 6*/             ThisSum += A[ k ]; /* sum of A[ i ] to A[ j ] */
    /* 7*/         if ( ThisSum > MaxSum )
    /* 8*/             MaxSum = ThisSum; /* update max sum
    /* 9*/     } /* end for-j and for-i */
    return MaxSum;
}
```

Detailed analysis
is your first
homework.

$$T(N) = O(N^3)$$

a1
a1 a2
a1 a2 a3
a1 a2 a3 a4
a1 a2 a3 a4 a5

a2
a2 a3
a2 a3 a4
a2 a3 a4 a5

a3
a3 a4
a3 a4 a5

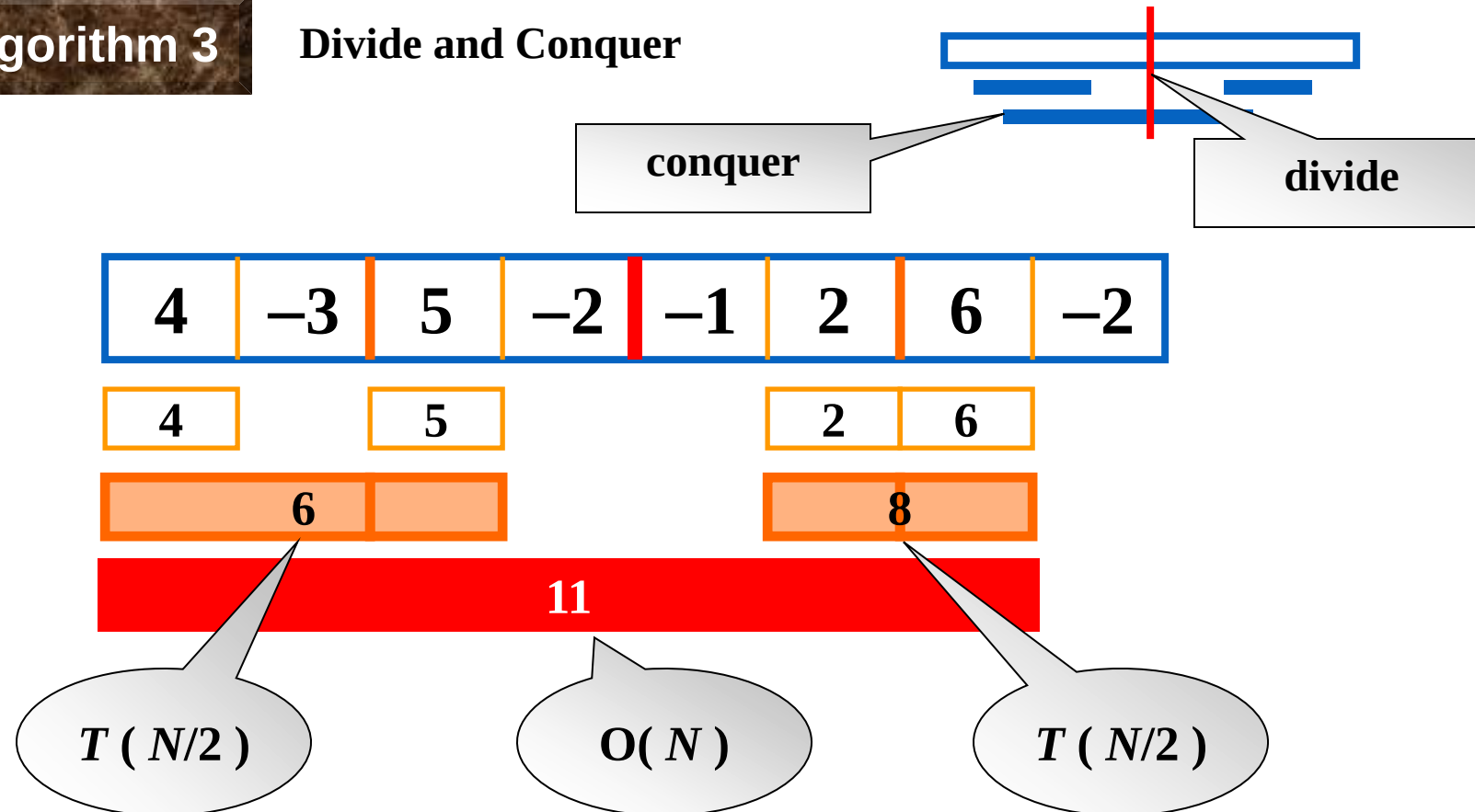
Algorithm 2

```
int MaxSubsequenceSum ( const int A[ ], int N )
{
    int ThisSum, MaxSum, i, j;
    /* 1*/ MaxSum = 0; /* initialize the maximum sum */
    /* 2*/ for( i = 0; i < N; i++ ) { /* start from A[ i ] */
    /* 3*/     ThisSum = 0;
    /* 4*/     for( j = i; j < N; j++ ) { /* end at A[ j ] */
    /* 5*/         ThisSum += A[ j ]; /* sum from A[ i ] to A[ j ] */
    /* 6*/         if ( ThisSum > MaxSum )
    /* 7*/             MaxSum = ThisSum; /* update max sum */
        } /* end for-j */
    } /* end for-i */
    /* 8*/ return MaxSum;
}
```

$$T(N) = O(N^2)$$

Algorithm 3

Divide and Conquer



$$\begin{aligned}
 T(N) &= 2T(N/2) + cN, & T(1) &= O(1) \\
 &= 2[2T(N/2^2) + cN/2] + cN \\
 &= 2^k O(1) + ckN & \text{where } N/2^k &= 1 \\
 &= O(N \log N)
 \end{aligned}$$

The program
would be
discussed later.

Algorithm 4

On-line Algorithm

```

int MaxSubsequenceSum( const int A[ ], int N )
{
    int ThisSum, MaxSum, j;
    /* 1*/ ThisSum = MaxSum = 0;
    /* 2*/ for ( j = 0; j < N; j++ ) {
    /* 3*/     ThisSum += A[ j ];
    /* 4*/     if ( ThisSum > MaxSum )
    /* 5*/         MaxSum = ThisSum;
    /* 6*/     else if ( ThisSum < 0 )
    /* 7*/         ThisSum = 0;
    /* 8*/ } /* end for-j */
    return MaxSum;
}

```



At any point in time, the algorithm can correctly give an answer to the **subsequence** problem for the data it has already read.

$T(N) = O(N)$

A[] is scanned **once** only.

Running times of several algorithms for maximum subsequence sum (in seconds)

Algorithm		1	2	3	4
Time		$O(N^3)$	$O(N^2)$	$O(N \log N)$	$O(N)$
Input Size	$N = 10$	0.00103	0.00045	0.00066	0.00034
	$N = 100$	0.47015	0.01112	0.00486	0.00063
	$N = 1,000$	448.77	1.1233	0.05843	0.00333
	$N = 10,000$	NA	111.13	0.68631	0.03042
	$N = 100,000$	NA	NA	8.0113	0.29832

Note: The time required to read the input is not included.



本章作业

Proof and compute.

- 证明关系

- $3n^2 + 4n^3 = O(n^3)$

- 求解线性递归关系:

- $$\begin{cases} x_{n+1} = 3x_n + 10x_{n-1}, n \geq 1 \\ x_0 = 2, x_1 = 3 \end{cases}$$

06

算法设计范式总览



分治法：分而治之的策略

将原问题拆分为 **独立子问题**，递归求解后合并结果；子问题规模缩小至常数时直接解决。归并排序、快速排序、二分查找皆属此列；复杂度通常呈 $n \log n$ 级别，体现“分解—解决—合并”三步节奏的强大威力。

基本技巧 Basic technique

先三分天下，各个击破；后囊括四海，并吞八荒

Divide into sub-questions; merge them back

主定理 Master Theorem

分治法复杂性通用公式

A formula for the complexity of DC

应用实例 Application

- 二分查找：有序序列，中点出发，大则向左，小则向右
- Binary Search: (in a sorted array) comparing with mid number, do binary search in left if its larger, do binary search in right if its smaller.
- 归并排序：递归排序，合二为一
- Merge Sort: recursively sort subarrays; merge them into one
- 矩阵乘法：减少递归数，降低复杂度
- Matrix Multiplication: decrease 8 times of recursive calls to 7, by reducing sub-matrix multiplication from 8 to 7
- 乘法优化：减少递归数，降低复杂度

```

--good_fib(10)
|--good_fib(9)
|  |--good_fib(8)
|  |  |--good_fib(7)
|  |  |  |--good_fib(6)
|  |  |  |  |--good_fib(5)
|  |  |  |  |  |--good_fib(4)
|  |  |  |  |  |  |--good_fib(3)
|  |  |  |  |  |  |  |--good_fib(2)
|  |  |  |  |  |  |  |  |--good_fib(1)
|  |  |  |  |  |  |  |  |  |-- return (1, 0)
|  |  |  |  |  |  |  |  |  |-- return (1, 1)
|  |  |  |  |  |  |  |  |  |-- return (2, 1)
|  |  |  |  |  |  |  |  |  |-- return (3, 2)
|  |  |  |  |  |  |  |  |  |-- return (5, 3)
|  |  |  |  |  |  |  |  |  |-- return (8, 5)
|  |  |  |  |  |  |  |  |  |-- return (13, 8)
|  |  |  |  |  |  |  |  |  |-- return (21, 13)
|  |  |  |  |  |  |  |  |  |-- return (34, 21)
|  |  |  |  |  |  |  |  |  |-- return (55, 34)
|  |  |  |  |  |  |  |  |  |-- return (55, 34)|

```

动态规划：记忆化最优子结构

适用于 **重叠子问题** 且全局最优包含子问题最优的场景；用表格存储中间结果，避免重复计算。背包问题、Floyd-Warshall 全源最短路径、HMM 解码均为经典应用；状态定义与转移方程是建模的核心突破口。

贪心算法：局部最优期望全局最优

每步选择当前看起来最佳选项，不回溯；若问题满足贪心选择性质，可得多项式时间精确解。算法简洁高效，易于工程实现。



Prim / Kruskal
最小生成树



Dijkstra
单源最短路径



Huffman
编码算法

回溯与分支限界：系统搜索解空间

回溯法 (Backtracking)



以深度优先枚举候选解，配合剪枝跳过无效子树。适用于排列组合问题。
如：八皇后、数独。

分支限界 (Branch & Bound)

9 20

采用宽度优先或优先队列，用上下界估计剪枝。适用于最优化问题。
如：旅行商问题。

06

高级算法思想与前沿趋势



启发式与随机算法：NP难的实用解

启发式算法

算法

模拟退火、遗传算法等借助自然隐喻在巨大解空间中快速逼近优解。以“可接受精度”换取“可承受时间”。

随机算法



随机化快速排序、蒙特卡罗积分等通过引入随机性降低最坏概率。同样以“概率”换取“效率”。

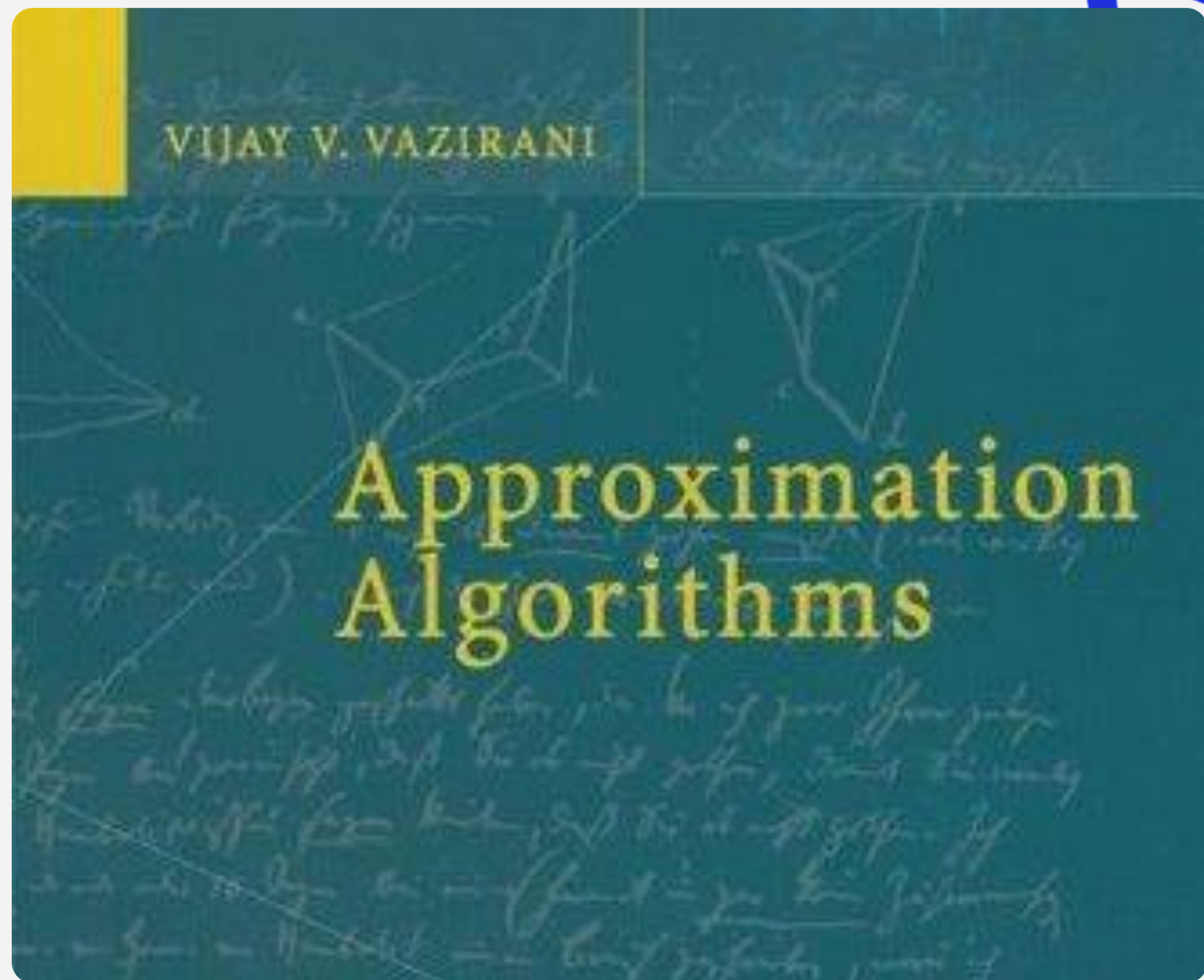
近似算法：带保证的误差界限

对无法多项式求解的 NP 难问题，设计近似算法，使其解与最优解之比不超过 **常数因子**。为工业界提供“即插即用”的理论保障。

顶点覆盖: 2-近似算法

TSP (三角不等式): 1.5-近似算法

装箱问题: AFPTAS 方案





自动算法设计：AI 反哺算法创作

借助神经架构搜索、程序合成与大模型代码生成，计算机可自动发现新奇算法或优化现有实现；[AlphaCode](#)、[Codex](#)在竞赛编程中已显身手，标志“算法的算法”正从元级研究走向实用工具，开启自我进化的新纪元。

未来展望：算法与社会的共生

随着量子计算、边缘计算与隐私计算兴起，算法设计将兼顾硬件异构、能耗限制与伦理合规；可解释性、公平性与低碳足迹成为新标准。



算法不再只是工具，而是社会技术系统的一部分，与人类价值持续对话、协同演进。

感谢您的观看

Thank you.

